

FITTRACK AI

Personalized Fitness Recommendations Powered by AI

PHASE 2 REQUIREMENT ANALYSIS

System Requirements Documentation

Document Date: September 28, 2026

Status: Requirements Analysis Phase

Table of Contents

1. Functional Requirements

2. Non-Functional Requirements

3. User Roles and Use Cases

4. System Architecture Overview

1. Functional Requirements

Functional requirements define what the FitTrack AI system must do. These requirements describe specific system features, functions, and behaviors that enable users to accomplish their fitness tracking and analysis objectives.

1.1 User Registration

The system must allow new users to create an account. Users can register by providing their name, email address, and password. The system must validate that the email is unique across all registered users and reject duplicate email addresses. Passwords must be securely hashed using bcrypt.js before storage in the database to ensure user security and privacy.

1.2 User Login

Registered users can authenticate by logging in with their email and password. The system must validate the provided credentials against stored user data. Upon successful authentication, the system generates a JWT

(JSON Web Token) that grants the user access to protected endpoints. This token must be included in subsequent API requests to verify user identity.

1.3 Protected User Profile Access

Authenticated users must be able to access their profile information. Profile endpoints must be protected and require valid JWT authentication. Users can only access their own profile data, not other users' information. The system ensures that profile-related operations are restricted to authorized users.

1.4 Workout Management

Authenticated users must have full control over their workout records through complete CRUD operations. Users can create new workout entries, view all their personal workouts, retrieve specific workouts by ID, update workout details, and delete outdated records. Each workout entry includes:

Workout name (exercise type)

Category (e.g., chest, legs, cardio, back)

Duration (in minutes)

Calories burned

Workout date and time

1.5 Workout Search and Filtering

The system must provide search and filtering capabilities to help users quickly locate specific workouts from their history. Users can filter workouts by workout name (exercise type), category (muscle group), and date. These search operations help users analyze their training patterns and identify specific workout sessions.

1.6 AI Workout Recommendation

The system must provide an endpoint that accepts user fitness parameters including age, fitness goal (e.g., weight loss, muscle gain, endurance), and experience level (beginner, intermediate, advanced). The system sends these parameters to Google Gemini AI, which analyzes the user profile and returns personalized workout recommendations tailored to the individual's specific situation.

1.7 AI Fitness Insights

The system must provide an endpoint that accepts user fitness statistics including total number of workouts completed, average workout duration, and total calories burned. The system sends these statistics to Google Gemini AI, which analyzes the data and returns meaningful fitness insights regarding the user's progress, fitness trends, and performance patterns.

1.8 API Response Handling

All API endpoints must return structured JSON responses with consistent formatting. Responses must include appropriate HTTP status codes (200 for success, 400 for client errors, 500 for server errors) and clear messages indicating the result of the operation. Data responses must be properly formatted for easy client-side consumption.

1.9 Validation and Error Handling

The system must validate all user input before processing. Input validation must include checking data types, required fields, and value ranges. Appropriate error messages must be returned when validation fails. The

system must handle errors gracefully and provide users with clear information about what went wrong and how to correct it.

1.10 Functional Requirements Table

Req ID	Requirement	Description	Priority
FR-001	User Registration	Users create account with name, email, password. Email must be unique. Password must be hashed using bcrypt.js.	High
FR-002	User Login	Users log in with email/password. System validates credentials and generates JWT token upon successful authentication.	High
FR-003	Profile Access	Authenticated users access their profile. Routes protected with JWT authentication. Users can only access own profile.	High
FR-004	Add Workout	Users create new workout entries with name, category, duration, calories burned, and date.	High
FR-005	View Workouts	Users view all personal workouts or retrieve specific workout by ID.	High
FR-006	Update Workout	Users update existing workout details with corrected or new information.	High
FR-007	Delete Workout	Users delete outdated or incorrect workout records from their history.	High
FR-008	Search Workouts	Users filter workouts by name, category, and date for quick retrieval and analysis.	High
FR-009	AI Recommendation	System accepts age, fitness goal, experience level; sends to Gemini AI; returns personalized workout recommendation.	High
FR-010	AI Fitness Insights	System accepts workout statistics; sends to Gemini AI; returns fitness insights and progress analysis.	High
FR-011	API Response	All endpoints return structured JSON with appropriate HTTP status codes and clear messages.	High
FR-012	Validation	System validates all input; provides clear error messages for validation failures.	High

2. Non-Functional Requirements

Non-functional requirements specify how the system should perform. These requirements focus on system quality attributes including security, reliability, performance, and maintainability.

Requirement	Description
Security	The system must implement industry-standard security practices. All sensitive operations must be protected, and data must be transmitted securely. The system must prevent unauthorized access and protect against common security vulnerabilities.
Authentication	JWT (JSON Web Tokens) must be used for stateless authentication. All protected endpoints require valid JWT tokens. Tokens must have appropriate expiration times and should be validated on each request.
Password Protection	Passwords must be hashed using bcrypt.js before storage in the database. Plain-text passwords must never be stored or logged. Password hashing must include appropriate salt rounds for security.
Data Privacy	User fitness data must remain confidential and accessible only to the data owner. The system must implement user isolation so users can only access their own workout records and personal information.
Performance	API endpoints must respond within reasonable timeframes. Database queries should be optimized for fast retrieval. The system must handle typical user loads without significant degradation in response times.
Reliability	The system must maintain data integrity and consistency. Operations must be atomic when necessary. The system should handle errors gracefully without losing user data. Recovery mechanisms should exist for potential failures.
Maintainability	Code must be well-organized and follow consistent coding standards. Modules should have clear separation of concerns. The codebase should be easily understandable for future maintenance and updates.
Scalability	The system architecture must support growth in users and data. Database design should accommodate increasing data volumes. API design should allow for future feature additions without major refactoring.
Usability	API endpoints must be intuitive and follow REST conventions. Responses must be consistent and easy to parse. Documentation must clearly explain how to use each endpoint.
Input Validation	All user inputs must be validated before processing. Invalid data must be rejected with clear error messages. Input length, type, and format must be verified against expected values.
Error Handling	Errors must be handled gracefully at all system levels. Error responses must include meaningful information for debugging. Errors should not expose sensitive system information to users.
API Consistency	All API endpoints must follow consistent naming conventions and response structures. Status codes must follow HTTP standards. Request/response formats must be uniform across the entire API.

3. User Roles and Use Cases

3.1 System Roles

Authenticated User

The primary user of the FitTrack AI system. An Authenticated User is any individual who has successfully registered an account and logged in. This user has access to all fitness tracking features and personalization

capabilities provided by the system. Their account is protected by JWT authentication, ensuring that only authorized users can access their personal fitness data.

3.2 Authenticated User Capabilities

Register and Create Account

Create a new account using name, email, and password.

Login to System

Authenticate using email and password to access the system.

View Profile Information

Access personal profile details through protected endpoints.

Manage Workouts

Add new workout records, view existing workouts, update workout details, and delete workouts.

Search Workouts

Filter and retrieve workouts by name, category, and date.

Request AI Workout Recommendations

Submit fitness profile (age, goal, experience level) to receive personalized AI-generated workout recommendations.

Request AI Fitness Insights

Submit fitness statistics to receive AI-generated analysis of fitness progress and performance insights.

3.3 Use Cases

ID	Use Case	Actor	Description	Expected Result
UC-001	Register	New User	User submits registration details (name, email, password) to create new account.	Account created; password hashed; user can login.
UC-002	Login	User	User provides email and password for authentication.	Credentials validated; JWT token issued; user authenticated.
UC-003	View Profile	User	Authenticated user requests their profile information.	Profile data displayed; access verified via JWT.
UC-004	Add Workout	User	User submits workout data (name, category, duration, calories, date).	Workout validated; stored in database; success confirmation.
UC-005	View Workouts	User	User requests all workouts or specific workout by ID.	Workout data retrieved; displayed in JSON format.
UC-006	Update Workout	User	User modifies existing workout details.	Workout updated; changes stored; confirmation returned.

UC-007	Delete Workout	User	User removes a workout record from history.	Workout deleted; database updated; success message.
UC-008	Search Workouts	User	User filters workouts by name, category, and/or date.	Matching workouts retrieved; filtered results displayed.
UC-009	AI Recommendation	User	User submits fitness profile to receive AI recommendation.	Gemini AI generates personalized workout recommendation.
UC-010	AI Insights	User	User submits fitness statistics for analysis.	Gemini AI generates meaningful fitness insights.

4. System Architecture Overview

4.1 Architecture Description

FitTrack AI is built using a modern, scalable REST API architecture implemented with Node.js and Express.js. The system follows a multi-layered architectural pattern that separates concerns and enables maintainability.

4.2 Architecture Layers

Presentation Layer (API Endpoints)

Express.js REST API endpoints serve as the interface for client applications. Endpoints implement HTTP GET, POST, PUT, DELETE methods following RESTful conventions. All responses are formatted as JSON with consistent structure.

Authentication & Security Layer

JWT middleware validates authentication tokens on protected routes. Passwords are hashed using bcrypt.js. Security middleware protects against common vulnerabilities and enforces authentication requirements.

Business Logic Layer

Core application logic implements user registration, workout management, search functionality, and AI integration. This layer contains all business rules and data processing logic.

Data Access Layer (Mongoose/MongoDB)

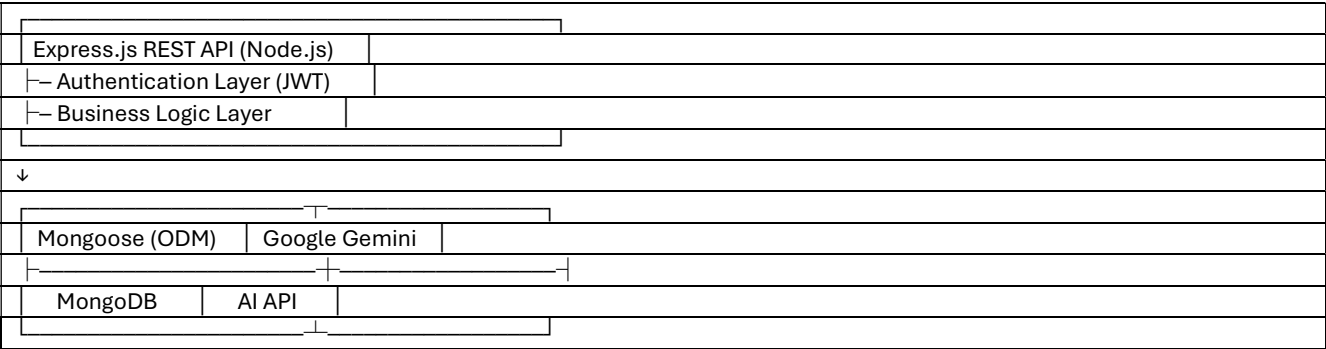
Mongoose ODM provides an abstraction layer over MongoDB. Data models define structure and validation for users, workouts, and related data. Database queries are optimized for performance.

External Integration Layer

Google Gemini API integration enables AI-powered recommendations and insights. External API calls are made asynchronously to avoid blocking main application threads.

4.3 Architecture Diagram

FitTrack AI System Architecture	
Client Application (Web/Mobile)	
↓ HTTP REST Requests ↓	



4.4 Key Architectural Principles

Separation of Concerns

Each layer has distinct responsibilities. Presentation, business logic, data access, and external integration are separate.

RESTful Design

Endpoints follow REST conventions with proper HTTP methods and status codes.

Security First

Authentication, encryption, and data validation are implemented throughout.

Scalability

Architecture supports growth in users and data without major changes.

4.5 Requirements Traceability

Phase 2 – Requirement Analysis has comprehensively documented all functional and non-functional requirements for FitTrack AI. Each requirement has been analyzed and validated against project objectives. The requirements form the foundation for the next phases of project development. All functional requirements (FR-001 through FR-012) have been mapped to corresponding use cases, ensuring complete traceability from requirements to implementation.